Surfacing Facetwork Workflows via MCP

Status: design exploration **Scope:** how Facetwork should expose its distributed workflow runner as a Model Context Protocol (MCP) server, how LLM agents such as Claude discover and invoke the resulting capabilities, and how FFL's mixin model can be leveraged to produce composable, machine-readable tool descriptions rather than hand-written prose.

---

1. Context and motivation

Facetwork today is a distributed workflow engine: FFL defines workflows, a registry of runners executes them, and operators interact through a dashboard, CLI, and an existing MCP server (`python -m facetwork.mcp`) that exposes compiler and management tools.

The current MCP server is operator-facing: compile FFL, validate, inspect the runner fleet, query PostGIS. It does not expose the **workflows themselves** as callable capabilities.

The question this document addresses is whether — and how — we should surface workflows as first-class MCP tools so that an LLM agent can:

1. **Discover** what workflows exist.
2. **Inspect** their parameters and contracts.
3. **Invoke** them over the distributed runner substrate.
4. **Chain** several invocations in succession to answer a higher-level user request.

If this works, Facetwork becomes an **AI-native control layer**: the LLM is a first-class caller of the runner, not a parallel scripting path.

---

2. How agents discover MCP functionality

MCP is a JSON-RPC 2.0 protocol. Discovery is explicit and has three primary endpoints.

2.1 `initialize`

The client (Claude Code, Cursor, a custom host) opens a connection and sends:

```
{
  "jsonrpc": "2.0", "id": 0, "method": "initialize",
  "params": { "protocolVersion": "2024-11-05", "capabilities": { ... } }
}
```

The server responds with its own capabilities — which of `tools`, `resources`, `prompts`, `logging`, `experimental` it supports. This is capability negotiation; the client will not call endpoints the server didn't advertise.

2.2 `tools/list`

```
{ "jsonrpc": "2.0", "id": 1, "method": "tools/list" }
```

Response lists every tool with `name`, `description`, and `inputSchema` (a JSON Schema). Each tool looks like:

```
{
  "name": "run_workflow",
  "description": "Start a Facetwork workflow run. Returns a run_id immediately; poll get_run_
  "inputSchema": {
    "type": "object",
    "properties": {
      "workflow": { "type": "string" },
      "params":   { "type": "object", "additionalProperties": true }
    },
    "required": ["workflow"]
  }
}
```

Servers that support pagination return `nextCursor`; servers that support dynamic registration emit a **`notifications/tools/list_changed`** message and the client re-fetches.

2.3 `resources/list` and `resources/read`

Resources are *readable* things addressed by URI — FFL source, run logs, prior outputs. The client calls `resources/list` to enumerate and `resources/read` to pull content:

```
{ "uri": "facetwork://workflows/osm.workflows.sourced.BicycleRoutesPostGIS",
  "name": "BicycleRoutesPostGIS",
  "description": "FFL source of the PostGIS-backed bicycle routes workflow.",
  "mimeType": "text/x-ffl" }
```

2.4 `prompts/list`

Prompt templates — pre-authored LLM prompts the server makes available. Useful for canned domain-specific starting points (e.g. "Analyze a PostGIS-imported region and summarize its road network").

2.5 What ends up in Claude's context

When Claude Code connects an MCP server, it calls `tools/list`, maps each tool to the Claude API's `tools` parameter schema, and appends them to its built-in tools (Grep, Read, Bash, …). From the model's perspective the provenance is invisible: a tool is a tool is a tool. Names get prefixed (`mcp__<server>__<tool>`) to avoid collisions.

Claude Code also supports **deferred tools**: to keep the initial tool list small when many servers are connected, it sends only names and short descriptions up-front, and the model fetches full schemas on demand via `ToolSearch`. This is an optimization, not part of MCP itself, but it matters for the scaling discussion in §5.

---

3. Why built-in tools "just work" and custom tools often don't

Claude uses `Grep` with no hand-holding because it has seen ripgrep/grep in its training corpus hundreds of thousands of times. The MCP `description` field and JSON Schema don't *teach* it grep; they merely:

- **Constrain the surface** (what flags and params are exposed) — the `Grep` tool is a curated wrapper, not raw grep. The model literally cannot pass flags that aren't in the schema.
- **Supply harness-specific guidance** — "ALWAYS use Grep, never shell out to `rg`"; "literal braces need escaping"; "use `multiline` for cross-line patterns." This is what the description field is for.

Everything else — regex syntax, when `-B` vs `-A` is appropriate, how to structure alternations — comes from pre-training.

For a novel tool like `BicycleRoutesPostGIS`, **there is no pre-training**. The LLM has never seen that name, that workflow, or the precise semantics of its parameters. The entire documentation burden shifts into the MCP tool metadata. Concretely:

Source	Built-in tools	Custom (Facetwork) tools
Semantics	Pre-training fills in	Must be in `description`
Parameter meaning	Pre-training + schema	Per-property `description` in the JSON Schema
Valid values / enums	Pre-training + examples	Schema `enum` (or described in prose, which is weaker)
When to use	Pre-training + description	Description only
Error recovery hints	Pre-training + tool_result error text	`tool_result` error text only

This asymmetry is the core reason MCP-server description quality is everything. Claude behaves well with well-documented MCP tools and dismally with poorly documented ones, for reasons that have nothing to do with model capability.

3.1 Tactics that compound

Seven tactics, each multiplying the effect of the others:

1. **Prose descriptions that read like terse reference docs.** Not "runs a workflow" but "Runs the PostGIS-backed bicycle-route extraction over a Geofabrik region. Async — returns `run_id` immediately; poll `get_run_status` until `status == 'completed'`. Typical runtime 5–40 minutes."
2. **Per-property descriptions in `inputSchema`.** JSON Schema lets every property carry its own `description`. Use it.
3. **Typed enums over prose enums.** If `region` must be one of the regions in `osm_import_log`, encode that as a JSON Schema `enum`. Then the model cannot pass an invalid region, period.
4. **Leverage patterns the model already knows.** Async/polling, idempotency keys, pagination cursors, JSON Schema conventions. Shape novel tools around these so pre-training can carry weight in the surrounding behavior.
5. **Expose discovery tools.** `list_workflows`, `describe_workflow(name)` — a human's `--help` equivalent.
6. **Resources and prompts as out-of-band docs.** FFL source as a readable resource; prior successful invocations as readable examples; canned prompt templates for common domain tasks.
7. **Error messages that teach.** `"region 'xyz' is not in osm_import_log; valid: [ ... ]"` corrects the model on the next turn. `"500 Internal Error"` wastes turns.

---

4. Mixins as composable description

FFL already has mixins: a workflow can be composed as `with FacetA() with FacetB()`. Today the composition is purely structural — it wires facets together. We can extend it to carry documentation.

4.1 The problem with monolithic prose

To expose `BicycleRoutesPostGIS` today, documentation would go in a single docstring or a sibling YAML file, along the lines of:

```
/** Extracts bicycle routes from PostGIS, computes statistics, and renders a map.
 *
 *  This workflow is async — it creates a run and returns a run_id immediately.
 *  Poll get_run_status(run_id) until status == "completed". Typical runtime
 *  5–20 minutes depending on region size.
 *
 *  Parameters:
 *    postgis_url — PostgreSQL connection URL. Must point at an OSM-imported
 *                  PostGIS database with osm_nodes and osm_ways tables.
 *    region      — Geofabrik region name (e.g. "Liechtenstein", "berlin").
 *                  Must match a region previously imported to PostGIS;
 *                  check osm_import_log.region for valid values.
 *
 *  Side effects:
 *    - Reads from osm_nodes and osm_ways tables
 *    - Writes map PNG and route GeoJSON under AFL_LOCAL_OUTPUT_DIR/<run_id>/
 *
 *  Returns: map_path (PNG), route_count, total_km
 */
workflow BicycleRoutesPostGIS(postgis_url: String = "...", region: String = "Liechtenstein")
  => (map_path: String, route_count: Long, total_km: Double) andThen { ... }
```

Problems with this pattern **at scale** (Facetwork already has dozens of workflows, and the design explicitly encourages many more):

- **Drift.** Every workflow restates async/polling boilerplate. Any change has to be hand-propagated.
- **Weak typing.** The `region` enum is documented in prose but not enforced in the schema. The model must parse English to know it needs to consult `osm_import_log`.
- **Duplication.** `postgis_url`'s description will be copy-pasted across 12+ workflows.
- **Lost structure.** The side-effect disclosure ("reads from `osm_nodes`/`osm_ways`") is buried in prose. It cannot be queried, filtered, or reasoned about structurally.
- **No composition dividend.** FFL already composes workflows from facets. The documentation model throws that composition away and starts over for every workflow.

4.2 Mixin-assembled description

Introduce a `describe { … }` clause on mixins. Each mixin carries typed metadata; the compiler composes the final MCP tool definition deterministically from the mixins the workflow uses.

```
namespace osm.mixins {

  /** Mixin: parameterizes a workflow with a Geofabrik-imported region. */
  facet WithRegion(region: String = "Liechtenstein") {
    describe {
      param region {
        summary: "Geofabrik region name"
        examples: ["Liechtenstein", "berlin", "switzerland"]
        enum_source: "postgis.osm_import_log.region"
        constraint: "must be previously imported to PostGIS"
      }
    }
  }

  /** Mixin: PostGIS data source. */
  facet PostGISSource(postgis_url: String = "postgresql://afl_osm:afl_osm_2024@afl-postgres:5
    describe {
      param postgis_url {
        summary: "PostgreSQL connection URL for the OSM-imported database"
        format: "postgres-url"
        secret: true
      }
      reads:   ["postgis:osm_nodes", "postgis:osm_ways"]
      touches: ["postgis:afl-postgres"]
    }
  }

  /** Mixin: this workflow is long-running and async. */
  facet LongRunning(estimated_minutes: Int) {
    describe {
      async: true
      typical_runtime_min: $.estimated_minutes
      invocation_pattern: "returns run_id; poll get_run_status"
    }
  }

  /** Mixin: writes output artifacts to the local output dir. */
  facet LocalOutputWriter {
    describe {
      writes: ["file:${AFL_LOCAL_OUTPUT_DIR}/${run_id}/"]
    }
  }
}
```

The workflow becomes:

```
/** Extracts bicycle routes from PostGIS, computes statistics, renders a map. */
workflow BicycleRoutesPostGIS
  with WithRegion()
  with PostGISSource()
  with LongRunning(estimated_minutes = 15)
  with LocalOutputWriter()
  => (map_path: String, route_count: Long, total_km: Double)
  andThen {
    routes = osm.Source.PostGIS.ExtractRoutes(
      source = #{"postgis_url": $.postgis_url, "region": $.region},
      route_type = "bicycle"
    )
    stats = osm.Routes.RouteStatistics(input_path = routes.result.output_path)
    map = osm.viz.RenderMap(geojson_path = routes.result.output_path,
                            title = "Bicycle Routes (PostGIS)", color = "#27ae60")
    yield BicycleRoutesPostGIS(
      map_path = map.result.output_path,
      route_count = routes.result.feature_count,
      total_km = stats.stats.total_length_km
    )
  }
```

The compiler emits the following MCP tool definition (composed from mixin `describe` blocks + the workflow's docstring):

```
{
  "name": "run_BicycleRoutesPostGIS",
  "description": "Extracts bicycle routes from PostGIS, computes statistics, renders a map.",
  "inputSchema": {
    "type": "object",
    "properties": {
      "region": {
        "type": "string",
        "default": "Liechtenstein",
        "description": "Geofabrik region name. Must be previously imported to PostGIS.",
        "enum": ["Liechtenstein", "berlin", "switzerland", "..."],
        "examples": ["Liechtenstein", "berlin"]
      },
      "postgis_url": {
        "type": "string",
        "format": "postgres-url",
        "description": "PostgreSQL connection URL for the OSM-imported database",
        "default": "postgresql://afl_osm:***@afl-postgres:5432/osm",
        "x-secret": true
      }
    }
  },
  "annotations": {
    "async": true,
    "typical_runtime_min": 15,
    "invocation_pattern": "returns run_id; poll get_run_status",
    "reads":   ["postgis:osm_nodes", "postgis:osm_ways"],
    "writes":  ["file:${AFL_LOCAL_OUTPUT_DIR}/${run_id}/"],
    "touches": ["postgis:afl-postgres"],
    "mixins":  ["WithRegion", "PostGISSource", "LongRunning", "LocalOutputWriter"]
  }
}
```

4.3 What mixins buy us (and what they cost)

Concern	Prose	Mixin-composed
Async/polling guidance	Duplicated per workflow	One `LongRunning` mixin
`region` enum	English prose, not enforced	Dynamic enum from `osm_import_log`
`postgis_url` docs	Copied 12×	Edit in one mixin
Side-effect disclosure	Prose or missing	Structured `reads` / `writes` / `touches`
Discovery by capability	Grep descriptions	Filter `mixins contains "PostGISSource"`
Drift risk between docs and code	High	Low — docs derive from code
Learning curve	None	Another clause to learn
Grammar/compiler work	Zero	`describe { }` clause, AST node, composer

Composition rules (choose deliberately)

Two mixins both describing the same parameter — what happens?

- **Error** — fail compilation; force the author to resolve by explicit override. *Pro:* no silent drift. *Con:* friction.
- **Last-wins** — the rightmost mixin in `with … with …` wins. *Pro:* zero friction. *Con:* surprising; silent bugs.
- **Merge** — union fields, error on conflict within a field. *Pro:* most forgiving. *Con:* hardest to specify precisely.

My recommendation: **error on conflict, explicit opt-in override** via `with PostGISSource() override describe { … }`. Fail loud.

Annotations are a convention, not standard MCP

The `annotations` object is a commonly-used MCP extension. Some clients read it, some don't. Design the server so the `description` and `inputSchema` carry the **load-bearing** information (what Claude needs to call the tool correctly), and let `annotations` be *upside* that richer clients can exploit. Do not put anything safety-critical only in `annotations`.

---

5. One MCP server, many workflows: discovery patterns

A Facetwork deployment will have tens to hundreds of workflows. How does one MCP server surface them all without drowning the LLM?

5.1 Pattern A — Flat: one MCP tool per workflow

`tools/list` returns `run_BicycleRoutesPostGIS`, `run_HikingTrailsPostGIS`, `run_MajorRoadsPBF`, and so on — one tool per workflow, each with its own typed schema.

When it works: small catalogs (< ~30 workflows), or subsets pinned for discoverability.

When it breaks: at scale the tool list dominates the context window; attention over hundreds of tools degrades discoverability. The LLM is also more likely to invent plausible-sounding tool names that don't exist.

Subvariant — tag-filtered flat: at `initialize`, the client passes a filter (namespace, tag, mixin) and the server only registers matching workflows. Good fit for multi-tenant setups where different users see different catalogs.

5.2 Pattern B — Catalog + generic invoker *(recommended default)*

Expose a small, fixed tool set that gives Claude a search-and-execute loop over a dynamic registry:

Tool	Purpose
`list_workflows(query?, namespace?, mixin?, tag?)`	Return matching workflows: `{name, summary, tags, mixins}`.
`describe_workflow(name)`	Return full JSON Schema + annotations for one workflow.
`run_workflow(name, params)`	Start a run; return `{run_id, status}`.
`get_run_status(run_id)`	Return `{status, progress, result?, error?}`.
`cancel_run(run_id)`	Cancel a running workflow.
`list_runs(filter?)`	Inspect recent/active runs (optional, useful for observability).

Structurally identical to how Claude Code itself works: **Glob** finds candidates, **Read** loads specifics, **Bash/Edit** act, results come back. Same loop, over workflows. Five tools cover a catalog of any size.

Tradeoffs:

- *Pro:* scales without bound; workflow registry can be dynamic; filter-by-mixin gives Claude capability-based discovery that prose can't match.
- *Pro:* generic `run_workflow` handles per-workflow schemas without inflating the tool list.
- *Con:* `run_workflow`'s static `inputSchema` cannot enforce per-workflow params — the LLM must first call `describe_workflow` to learn the schema, then pass it correctly. **Mitigation:** `run_workflow` returns a descriptive `422`-style error listing the actual required params if the LLM skips the `describe_workflow` step; the error teaches on the next turn.
- *Con:* an extra round-trip (`describe_workflow`) before every novel call. Usually worth it; cache hints via annotations can help.

5.3 Pattern C — Hybrid: pinned popular + catalog long tail

Workflows tagged `mcp_pin: true` get their own typed tools; everything else is reachable via the catalog.

- *Pro:* low-friction fast path for common workflows, no ceiling on total count.
- *Con:* two code paths, and the pin list needs governance.

This is what I'd evolve to once B is working. **Do not start here** — premature without the catalog foundation.

5.4 Decision matrix

Criterion	Flat (A)	Catalog (B)	Hybrid (C)
Catalog size tolerated	≤ 30	unbounded	unbounded
Tokens per session	high	minimal	moderate
Per-workflow typed schema at call site	yes	after `describe`	yes (pinned) / after describe (rest)
Workflow changes at runtime	`list_changed` required	seamless	`list_changed` for pins
Governance burden	low	low	medium (pin list)
Recommended starting point	only if catalog is small	yes	after B is proven

---

6. A worked example: prompt and tool-call trace

6.1 The user's prompt

> *"For the Berlin region, run the bicycle and hiking route analyses in parallel. When both finish, tell me which has more total km and show me the maps."*

The user names no workflows. Claude is expected to discover them.

6.2 Tool-call trace (Pattern B)

```
T1  list_workflows(query="bicycle routes", mixin="PostGISSource")
    ← [{name:"BicycleRoutesPostGIS",
        summary:"Extracts bicycle routes from PostGIS, computes statistics, renders a map",
        tags:["routes","postgis"]}]

T2  list_workflows(query="hiking", mixin="PostGISSource")
    ← [{name:"HikingTrailsPostGIS",
        summary:"Extracts hiking trails from PostGIS with elevation enrichment",
        tags:["routes","postgis"]}]

T3  describe_workflow("BicycleRoutesPostGIS")     ┐ parallel
    describe_workflow("HikingTrailsPostGIS")      ┘
    ← full JSON Schema each: region (enum from osm_import_log),
       postgis_url, returns map_path + total_km. Annotations:
       {async:true, typical_runtime_min:15, reads:[...]}.

T4  run_workflow("BicycleRoutesPostGIS", {region:"berlin"})   ┐ parallel
    run_workflow("HikingTrailsPostGIS",  {region:"berlin"})   ┘
    ← {run_id:"6618a4", status:"running"}
    ← {run_id:"6618a5", status:"running"}

T5  get_run_status("6618a4")   ┐ parallel
    get_run_status("6618a5")   ┘
    ← {status:"running", progress:0.20}
    ← {status:"running", progress:0.15}

    [Claude waits — polling cadence driven by annotations (~15 min);
     exact mechanism depends on harness long-running-tool support.]

T6  get_run_status("6618a4")   ┐ parallel
    get_run_status("6618a5")   ┘
    ← {status:"completed", result:{map_path:"/.../bike.png",
        route_count:1847, total_km:2310.4}}
    ← {status:"completed", result:{map_path:"/.../hike.png",
        trail_count:302,  total_km:488.7}}

T7  (text to user)
    "Bicycle routes: 2,310 km across 1,847 routes.
     Hiking trails: 489 km across 302 trails.
     Berlin has ≈4.7× more cycling than hiking distance.
     Maps: /.../bike.png, /.../hike.png"
```

6.3 What makes this work

This is the same loop shape Claude already uses for `Grep → Read → Edit`. Three design choices enable it:

1. **Descriptions prescribe the chain.** `run_workflow`'s description must explicitly say: "*If `status != 'completed'`, call `get_run_status(run_id)` to poll. Typical workflows take 5–40 minutes; do not*

retry `run_workflow` with the same params." Without this, Claude guesses the pattern — and often guesses wrong.
2. **`list_workflows` filters are rich.** Filtering by **mixin** is the critical lever: "PostGIS-backed route extractor" is three matches, not thirty.
3. **Parallel tool calls are cheap.** Claude can emit multiple `tool_use` blocks in one response. The MCP server must tolerate concurrent `run_workflow` and `get_run_status` calls. The trace above assumes it.

---

7. Long-running workflow patterns

Runs take minutes to hours. Three patterns for handling that over MCP:

7.1 Polling (shown in the trace)

- *Pro:* simple; works today with any MCP client.
- *Con:* wastes a few turns per run; cadence is heuristic.
- *When to use:* first cut. Nearly every system should start here.

7.2 Server-initiated notifications

MCP supports server-initiated messages. The server can emit a `run_completed` notification; a capable client surfaces it to the model.

- *Pro:* no polling waste; immediate handoff to the model when done.
- *Con:* requires client-side support; not all hosts implement it yet.

7.3 Streaming progress as partial tool results

`run_workflow` blocks and streams partial `tool_result` updates (`progress: 0.3`, `current_step: "RouteStatistics"`) until the final result.

- *Pro:* feels native to the LLM; no polling logic needed.
- *Con:* depends on streaming tool-use support in the client; handler-side complexity.

7.4 Recommendation

Start with **polling + prescriptive description** (§7.1). The description should include:

- whether the tool is async at all,

- the typical runtime range,
- what value of `status` indicates terminal success/failure,
- what cadence to poll at (e.g. "*every 30s for the first 5 minutes, then every 2 minutes*").

This lets Claude apply good judgment even when it has no pre-training. Move to notifications or streaming only when polling's overhead becomes a measurable problem.

---

8. Cross-cutting concerns

8.1 Security and authorization

Running a workflow is a side-effecting action: compute is spent, databases are written, artifacts appear on disk. The MCP server must answer:

- **Who is the caller?** MCP hosts typically pass a bearer/API key; Facetwork should bind that key to a principal and record it on every run.
- **What can each principal run?** A per-principal allowlist (by namespace, tag, or explicit workflow name) prevents accidental or hostile invocation of expensive workflows.
- **Rate limiting and quotas.** Without these, a single hallucinatory chain could exhaust the runner fleet.
- **Secret redaction.** Parameters marked `secret: true` (§4.2) must be redacted in logs and in `describe_workflow` output — defaults can show `"***"` only.

8.2 Schema and workflow versioning

Workflow definitions change. Questions to answer explicitly:

- Is a run tied to a **workflow version** at submission time? (Recommended: yes — the server snapshots the compiled definition.)
- Does `describe_workflow` return the latest version or a specific revision?
- If a workflow is edited while runs are in flight, do in-flight runs fail, complete on the old version, or both (new runs pick up the new version, old runs finish)?

Facetwork's runner architecture already stores compiled workflow graphs; extending this to expose a version identifier on each run makes the semantics unambiguous to both humans and LLMs.

8.3 Idempotency and retries

An LLM that times out and retries `run_workflow` with the same params should not start a second run of the same work. Offer an optional `idempotency_key` on `run_workflow` — if a run with that key already exists, return its `run_id` instead of starting a new one. This is a well-known pattern the LLM has seen many times and will use correctly if told it exists.

8.4 Observability

`get_run_status` should return structured progress, not just a free-text message. Fields like `current_step`, `completed_steps`, `total_steps`, `last_log_line`, `estimated_completion_at` let the LLM answer "how close is it?" without scraping logs. This also lets the harness render meaningful UX.

8.5 Cost and time signals in annotations

Include **expected cost/time** in the MCP `annotations`. An LLM that knows `typical_runtime_min: 180` before calling can ask the user for confirmation rather than silently committing the user to a three-hour wait. Do not surface this only in prose — put it in structured annotations.

8.6 Human-in-the-loop gating

Some workflows should *require* human confirmation before execution (destructive data moves, large compute). Model this with an `annotations.requires_confirmation: true` flag that the host uses to prompt the user before invocation. Do not rely solely on the LLM's judgment.

---

9. What to prototype first

Concretely, in priority order:

1. **Implement `list_workflows`, `describe_workflow`, `run_workflow`, `get_run_status`** on the existing `facetwork.mcp` server. Use the current workflow registry in MongoDB as the backing store.
2. **Emit a prescriptive description on `run_workflow`** that tells the model how to poll. Test end-to-end with Claude Code and ordinary user prompts.
3. **Add `idempotency_key` and structured `get_run_status` fields** — these cost little and recover a lot of robustness.
4. **Prototype `describe` clauses on 3–4 mixins** (`WithRegion`, `PostGISSource`, `LongRunning`, `LocalOutputWriter`). Emit composed MCP tool definitions for one namespace (`osm.workflows.sourced`).

5. **A/B test** the prose vs. mixin-composed descriptions on the same set of user prompts. Measure: turns to completion, invalid-param failures, hallucinated workflow names. Expect: the typed `region` enum alone collapses one entire failure mode.
6. **Add authorization, versioning, and quotas** before exposing the server outside a controlled environment.

Pattern C (hybrid pinning) and streaming tool results are follow-ups, not first-cut. Resources and prompts are a natural second iteration once the core loop feels right.

---

10. Summary

- **Discovery is well-defined.** `tools/list` + `resources/list` + `prompts/list` give LLMs a structured view of a server's capabilities. `notifications/tools/list_changed` handles dynamic registration.
- **The pre-training gap is the real constraint.** Built-in tools like Grep work because the model has seen them; custom tools like `BicycleRoutesPostGIS` do not have that luxury. Description quality is destiny.
- **Mixins turn documentation into a composable asset.** A `describe { }` clause on FFL mixins lets typed, reusable fragments compose into MCP tool definitions the same way the workflow itself composes from facets. Docs cannot drift from code because they are derived from the same structure.
- **Pattern B (catalog + invoker) is the right default** for surfacing many workflows from one MCP server. It mirrors the Grep/Read/Edit loop Claude already executes well and scales to arbitrary catalog sizes.
- **Long-running semantics need explicit treatment.** Polling with a prescriptive description is the pragmatic starting point; streaming and notifications are later wins.
- **Security, versioning, idempotency, and observability** are non-negotiable and should be built in from the start, not bolted on.

The strategic point is that Facetwork's **structured composition model** — facets, mixins, typed schemas — turns into a durable advantage precisely because the consumers are language models. Prose descriptions are a lossy serialization of structure the compiler already has. Surfacing that structure directly to the MCP layer is the difference between an LLM that can use Facetwork effectively and one that keeps inventing workflows that don't exist.